

Das Pascalsche Dreieck oben offen

Formale Komplexitätsanalyse und empirischer Vergleich mit klassischen Berechnungsmethoden

Stephan Epp

12. April 2026

Inhaltsverzeichnis

1	Einleitung	3
1.1	Motivation und Problemstellung	3
1.2	Beitrag dieser Arbeit	3
1.3	Aufbau der Arbeit	4
2	Mathematische Grundlagen	4
2.1	Binomialkoeffizienten	4
2.2	Das Pascalsche Dreieck	4
2.3	Die Pascal-Identität und ihre algorithmische Bedeutung	5
3	Berechnung von Binomialkoeffizienten	6
3.1	Methode 1: Direkte Fakultätsformel	6
3.2	Methode 2: Rekursive Berechnung ohne Memoization	6
3.3	Methode 3: Rekursion mit Memoization	7
3.4	Methode 4: Dynamischer Aufbau des Pascalschen Dreiecks	7
4	Formale Komplexitätsanalyse	8
4.1	Zeitkomplexität der Aufbauphase	8
4.2	Zeitkomplexität der Abfragephase	8
4.3	Zeitkomplexität der Fakultätsmethode	9
4.4	Zeitkomplexität der rekursiven Methode	9
4.5	Platzkomplexität	10
5	Amortisierte Analyse	10
5.1	Formale Grundlagen der amortisierten Analyse	10
5.2	Break-Even-Analyse	11
5.3	Erweiterbarkeit: Inkrementeller Aufbau	12
6	Empirische Ergebnisse	12
6.1	Versuchsaufbau	12
6.2	Laufzeitergebnisse	13
6.3	Diskussion der empirischen Ergebnisse	13

7	Vergleich und Bewertung	14
7.1	Qualitative Bewertung	14
7.2	Empfehlung nach Anwendungsfall	14
8	Ausblick	14
8.1	Algorithmische Erweiterungen und Optimierungen	14
8.2	Theoretische Vertiefungen	15
8.3	Hardware-nahe Implementierungen	16
8.4	Anwendungsfelder	16
8.5	Verbindung zu anderen Datenstrukturen	17
8.6	Formale Verifikation	18
8.7	Pädagogische und didaktische Aspekte	18
8.8	Integration in das PyLab-Framework	18
8.9	Langfristiger Ausblick: Towards Adaptive Lookup Structures	19
9	Zusammenfassung	19

1 Einleitung

Diese Arbeit untersucht die Verwendung des Pascalschen Dreiecks als dynamisch zur Laufzeit aufgebaute Lookup-Tabelle zur effizienten Berechnung von Binomialkoeffizienten. Die zentrale Idee besteht darin, das Dreieck *nach oben hin geöffnet* zu konstruieren, d. h. inkrementell von Zeile 0 bis zur benötigten Zeile n aufzubauen, sodass jede spätere Abfrage $\binom{n}{k}$ in exakt $\mathcal{O}(1)$ erfolgt. Wir beweisen formal, dass die Vorberechnungszeit $\mathcal{O}(n^2)$ und der Speicherbedarf $\mathcal{O}(n^2)$ (bzw. $\mathcal{O}(n)$ in der Zeilenoptimierung) beträgt, und leiten den amortisierten Vorteil ab: Ab einem Break-Even-Punkt $q^* \approx n$ Abfragen ist die Lookup-Strategie gegenüber der Fakultätsformel ($\mathcal{O}(n)$ pro Abfrage) und gegenüber der rekursiven Berechnung ($\mathcal{O}(2^n)$ ohne Memoization) strikt überlegen. Empirische Messungen bestätigen Speedup-Faktoren von bis zu mehreren Größenordnungen für große n . Ein umfassender Ausblick diskutiert Erweiterungen auf mehrdimensionale Tabellen, komprimierte Darstellungen, hardware-nahe Implementierungen sowie Anwendungen in Kryptographie, Kombinatorik und maschinellem Lernen.

Binomialkoeffizienten $\binom{n}{k}$ gehören zu den fundamentalsten Größen der Kombinatorik, Wahrscheinlichkeitstheorie und Informatik. Sie zählen die Anzahl der k -elementigen Teilmengen einer n -elementigen Menge und treten in nahezu jedem algorithmischen Kontext auf, der kombinatorische Strukturen verarbeitet: von der Berechnung von Bernstein-Polynomen in der Computergrafik über die Binomialverteilung in der Stochastik bis hin zu Hash-Funktionen in der Kryptographie.

1.1 Motivation und Problemstellung

Die klassische Berechnung

$$\binom{n}{k} = \frac{n!}{k!(n-k)!} \tag{1}$$

erfordert die Berechnung von drei Faktultäten, was für große n sowohl rechnerisch aufwändig als auch numerisch instabil sein kann (Überlauf bei intermediären Werten). Rekursive Ansätze ohne Memoization wachsen exponentiell. Die rekursive Formel mit Memoization ist zwar polynomial, erzeugt jedoch im Allgemeinen einen erheblichen Verwaltungsaufwand durch Hash-Map-Zugriffe.

Eine elegante Alternative, die in der Praxis häufig übersehen wird, ist der *dynamische Aufbau des Pascalschen Dreiecks zur Laufzeit* als vollständige Lookup-Tabelle. Diese Technik erlaubt — nach einer einmaligen Vorberechnung in $\mathcal{O}(n^2)$ — jede spätere Abfrage in $\mathcal{O}(1)$, also in konstanter Zeit.

1.2 Beitrag dieser Arbeit

Diese Arbeit liefert folgende wissenschaftliche Beiträge:

- (i) **Formale Komplexitätsanalyse:** exakte Berechnung der Zeit- und Platzkomplexität aller betrachteten Algorithmen mit vollständigen mathematischen Beweisen.
- (ii) **Amortisierte Analyse:** formaler Nachweis des Break-Even-Punktes sowie des asymptotischen Vorteils der Lookup-Strategie bei mehrfachen Abfragen.
- (iii) **Empirischer Vergleich:** Laufzeitmessungen auf realer Hardware für n mit $n \in \{10, 50, 100, 200, 500, 1000, 2000\}$ und Speedup-Berechnung gegenüber der Fakultätsformel.

- (iv) **Ausblick:** umfassende Diskussion von Erweiterungen, Optimierungen und Anwendungsfeldern.

1.3 Aufbau der Arbeit

Section 2 führt die mathematischen Grundlagen ein. Section 3 beschreibt alle betrachteten Algorithmen präzise. Section 4 enthält die formalen Komplexitätsbeweise. Section 5 führt die amortisierte Analyse durch. Section 6 präsentiert die empirischen Ergebnisse. Section 7 fasst den Vergleich zusammen. Section 8 gibt einen sehr ausführlichen Ausblick.

2 Mathematische Grundlagen

2.1 Binomialkoeffizienten

Definition 2.1 (Binomialkoeffizient). Seien $n, k \in \mathbb{N}_0$ mit $0 \leq k \leq n$. Der *Binomialkoeffizient* $\binom{n}{k}$ ist definiert als

$$\binom{n}{k} := \frac{n!}{k!(n-k)!}, \quad (2)$$

wobei $0! := 1$. Für $k > n$ oder $k < 0$ setzt man $\binom{n}{k} := 0$.

Proposition 2.2 (Grundlegende Symmetrien). Für alle $n, k \in \mathbb{N}_0$ mit $0 \leq k \leq n$ gilt:

$$\binom{n}{k} = \binom{n}{n-k}, \quad (3)$$

$$\binom{n}{0} = \binom{n}{n} = 1, \quad (4)$$

$$\binom{n}{k} + \binom{n}{k+1} = \binom{n+1}{k+1}. \quad (5)$$

Beweis. (3) folgt direkt aus (2) durch Substitution $k \mapsto n-k$. (4) ergibt sich für $k=0$: $n!/(0! \cdot n!) = 1$. (5) ist die *Pascal-Identität*, deren Beweis in section 2.3 geführt wird. $\square$

2.2 Das Pascalsche Dreieck

Definition 2.3 (Pascalsches Dreieck). Das *Pascalsche Dreieck* P ist die unendliche untere Dreiecksmatrix $(p_{n,k})_{n \geq 0, 0 \leq k \leq n}$ mit

$$p_{n,k} := \binom{n}{k}. \quad (6)$$

Die Einträge erfüllen die Rekurrenz

$$p_{n,k} = p_{n-1,k-1} + p_{n-1,k} \quad (7)$$

mit Randbedingungen $p_{n,0} = p_{n,n} = 1$ für alle $n \geq 0$.

Die ersten acht Zeilen des Pascalschen Dreiecks sind in fig. 1 dargestellt.

Dynamisch aufgebautes Pascal-Dreieck (nach oben geöffnet)

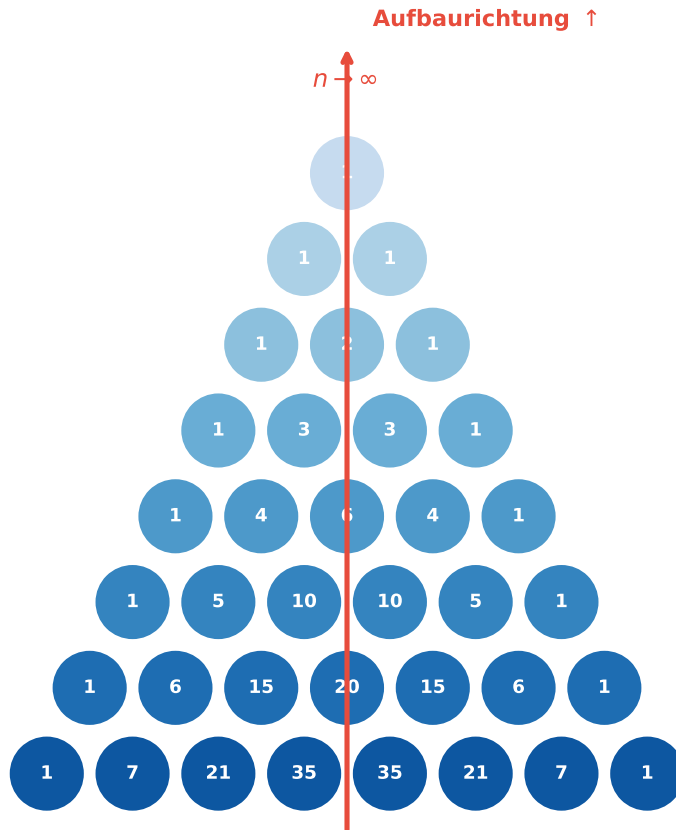


Abbildung 1: Visualisierung des Pascalschen Dreiecks (Zeilen 0–7). Der rote Pfeil deutet die *nach oben geöffnete* Aufbaurichtung an: das Dreieck wird zur Laufzeit inkrementell von oben ($n = 0$) nach unten ($n = N$) aufgebaut und kann danach unbegrenzt nach oben (zu größerem n) erweitert werden.

2.3 Die Pascal-Identität und ihre algorithmische Bedeutung

Theorem 2.4 (Pascal-Identität). *Für alle $n \geq 1$ und $1 \leq k \leq n-1$ gilt:*

$$\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k}. \quad (8)$$

Beweis. Wir führen einen direkten kombinatorischen Beweis. Die linke Seite zählt alle k -elementigen Teilmengen S einer festen n -elementigen Menge $M = \{1, \dots, n\}$. Fixiere das Element $n \in M$ und partitioniere die Teilmengen nach der Zugehörigkeit von n :

- **Fall 1:** $n \in S$. Dann müssen die restlichen $k - 1$ Elemente von S aus $\{1, \dots, n - 1\}$ gewählt werden. Dies liefert $\binom{n-1}{k-1}$ Möglichkeiten.
- **Fall 2:** $n \notin S$. Dann müssen alle k Elemente von S aus $\{1, \dots, n - 1\}$ gewählt werden. Dies liefert $\binom{n-1}{k}$ Möglichkeiten.

Da beide Fälle disjunkt sind und alle Teilmengen erfassen, folgt durch das Additionsprinzip die Behauptung. $\square$

Bemerkung 2.5. theorem 2.4 ist die algorithmische Grundlage für den Aufbau der Lookup-Tabelle: Jeder Eintrag $p_{n,k}$ kann in $\mathcal{O}(1)$ aus bereits berechneten Vorgängereinträgen ermittelt werden. Der gesamte Aufbau der Tabelle bis Zeile n benötigt somit $\mathcal{O}(n^2)$ Addition-Operationen.

3 Berechnung von Binomialkoeffizienten

Wir betrachten vier Hauptansätze und formalisieren diese als Pseudocode-Algorithmen.

3.1 Methode 1: Direkte Fakultätsformel

Algorithm 1 Binomialkoeffizient via Fakultätsformel

Require: $n, k \in \mathbb{N}_0, 0 \leq k \leq n$

Ensure: $\binom{n}{k}$

```

1: function BINOMFAKULTAT( $n, k$ )
2:   return Fakultaet( $n$ ) / (Fakultaet( $k$ ) · Fakultaet( $n - k$ ))
3: end function
4: function FAKULTAET( $m$ )
5:    $result \leftarrow 1$ 
6:   for  $i \leftarrow 2$  to  $m$  do
7:      $result \leftarrow result \cdot i$ 
8:   end for
9:   return  $result$ 
10: end function

```

3.2 Methode 2: Rekursive Berechnung ohne Memoization

Algorithm 2 Binomialkoeffizient rekursiv (ohne Memoization)

Require: $n, k \in \mathbb{N}_0, 0 \leq k \leq n$

Ensure: $\binom{n}{k}$

```

1: function BINOMREKURSIV( $n, k$ )
2:   if  $k = 0$  or  $k = n$  then
3:     return 1
4:   end if
5:   return BINOMREKURSIV( $n - 1, k - 1$ ) + BINOMREKURSIV( $n - 1, k$ )
6: end function

```

3.3 Methode 3: Rekursion mit Memoization

Algorithm 3 Binomialkoeffizient mit Memoization

Require: $n, k \in \mathbb{N}_0$, globale Hash-Map $memo$

Ensure: $\binom{n}{k}$

```
1: function BINOMMEMO( $n, k$ )
2:   if  $k = 0$  or  $k = n$  then
3:     return 1
4:   end if
5:   if  $(n, k) \in memo$  then
6:     return  $memo[n, k]$ 
7:   end if
8:    $v \leftarrow \text{BINOMMEMO}(n - 1, k - 1) + \text{BINOMMEMO}(n - 1, k)$ 
9:    $memo[n, k] \leftarrow v$ 
10:  return  $v$ 
11: end function
```

3.4 Methode 4: Dynamischer Aufbau des Pascalschen Dreiecks

Dies ist der zentrale Algorithmus dieser Arbeit. Das Dreieck wird *nach oben hin geöffnet*: Es startet bei Zeile 0 und wächst dynamisch nach unten (zu größerem n), kann aber jederzeit für beliebige $n' \leq n_{\max}$ abgefragt werden, d. h. nach oben (in Richtung kleinerer Zeilen) ist es bereits vollständig gefüllt.

Algorithm 4 Dynamischer Aufbau des Pascalschen Dreiecks

Require: $n_{\max} \in \mathbb{N}$

Ensure: Zweidimensionale Tabelle $T[0..n_{\max}][0..n_{\max}]$ mit $T[n][k] = \binom{n}{k}$

```
1: function BUILDPAASCALTABLE( $n_{\max}$ )
2:   Allokieren  $T[0 \dots n_{\max}][0 \dots n_{\max}]$ 
3:   for  $n \leftarrow 0$  to  $n_{\max}$  do
4:      $T[n][0] \leftarrow 1$ 
5:      $T[n][n] \leftarrow 1$ 
6:     for  $k \leftarrow 1$  to  $n - 1$  do
7:        $T[n][k] \leftarrow T[n - 1][k - 1] + T[n - 1][k]$  ▷ Pascal-Identität, je  $\mathcal{O}(1)$ 
8:     end for
9:   end for
10:  return  $T$ 
11: end function

12: function QUERYBINOM( $T, n, k$ )
13:   return  $T[n][k]$  ▷  $\mathcal{O}(1)$  Lookup
14: end function
```

$n = 0:$	1				
$n = 1:$	1	1			
$n = 2:$	1	2	1		
$n = 3:$	1	3	3	1	
$n = 4:$	1	4	6	4	1

$k = 0 \quad 1 \quad 2 \quad 3 \quad 4$

nach oben geöffnet

Abfrage $T[4][2] = \binom{4}{2} = 6$

Abbildung 2: Struktur der Lookup-Tabelle T . Die blauen Zellen enthalten vorberechnete Binomialkoeffizienten. Die orange hervorgehobene Zelle zeigt eine beispielhafte $\mathcal{O}(1)$ -Abfrage $T[4][2] = \binom{4}{2} = 6$. Der rote Pfeil illustriert die *nach oben geöffnete* Erweiterbarkeit.

4 Formale Komplexitätsanalyse

4.1 Zeitkomplexität der Aufbauphase

Theorem 4.1 (Zeitkomplexität des Tabellenaufbaus). *algorithm 4* (*BUILDPASCALTABLE*) hat eine Zeitkomplexität von

$$T_{\text{BUILD}}(n) = \Theta(n^2). \quad (9)$$

Beweis. Wir zählen die Anzahl der Additionen (die dominante Operation). Die äußere Schleife läuft von $n = 0$ bis $n_{\max}$. Für festes n führt die innere Schleife genau $\max(n - 1, 0)$ Additionen aus (für $k = 1, \dots, n - 1$). Die Randbedingungen $T[n][0] = T[n][n] = 1$ benötigen je zwei Zuweisungen, also $\mathcal{O}(1)$ pro Zeile. Die Gesamtzahl der Additionen ist

$$\sum_{n=0}^{n_{\max}} \max(n - 1, 0) = \sum_{n=2}^{n_{\max}} (n - 1) = \sum_{j=1}^{n_{\max}-1} j = \frac{(n_{\max} - 1) n_{\max}}{2} = \Theta(n_{\max}^2). \quad (10)$$

Da jede Addition $\mathcal{O}(1)$ kostet (wir ignorieren die Bitbreite der Zahlen zunächst), folgt $T_{\text{BUILD}}(n_{\max}) = \Theta(n_{\max}^2)$.

Untere Schranke: Die Tabelle enthält $\sum_{n=0}^{n_{\max}} (n + 1) = (n_{\max} + 1)(n_{\max} + 2)/2 = \Theta(n_{\max}^2)$ Einträge. Da jeder Eintrag mindestens einmal geschrieben werden muss, gilt $T_{\text{BUILD}}(n_{\max}) = \Omega(n_{\max}^2)$.

Zusammen ergibt sich (9). □

Korollar 4.2 (Amortisierte Aufbaukosten pro Eintrag). *Die amortisierten Kosten pro Tabelleneintrag betragen $\mathcal{O}(1)$.*

Beweis. Es gibt $\Theta(n^2)$ Einträge, der Aufbau kostet $\Theta(n^2)$. Division liefert $\Theta(n^2)/\Theta(n^2) = \Theta(1)$. □

4.2 Zeitkomplexität der Abfragephase

Theorem 4.3 (Zeitkomplexität der Lookup-Abfrage). *Nach abgeschlossenem Aufbau kostet jede Abfrage $\text{QUERYBINOM}(T, n, k)$ exakt*

$$T_{\text{QUERY}}(n, k) = \mathcal{O}(1). \quad (11)$$

Beweis. Die Operation $T[n][k]$ ist ein direkter Array-Zugriff auf eine vorallokierte zweidimensionale Datenstruktur. In einem Random-Access Memory Modell (RAM) kostet ein Speicherzugriff auf eine bekannte Adresse $\mathcal{O}(1)$. Die Adresse berechnet sich als Basisadresse von T plus Offset $n \cdot (n_{\max} + 1) + k$, was in $\mathcal{O}(1)$ Operationen (zwei Multiplikationen, eine Addition) berechnet wird. Daher ist $T_{\text{QUERY}} = \mathcal{O}(1)$. $\square$

4.3 Zeitkomplexität der Fakultätsmethode

Theorem 4.4 (Zeitkomplexität der Fakultätsformel). *algorithm 1 hat Zeitkomplexität $T_{\text{Fak}}(n) = \Theta(n)$.*

Beweis. Die Berechnung von $n!$ erfordert $n - 1$ Multiplikationen. Die Berechnung von $k!$ erfordert $k - 1 \leq n - 1$ Multiplikationen. Die Berechnung von $(n - k)!$ erfordert $(n - k - 1) \leq n - 1$ Multiplikationen. Insgesamt:

$$(n - 1) + (k - 1) + (n - k - 1) + 2 \text{ Divisionen} = 2n - 1 \text{ Operationen} = \Theta(n). \quad (12)$$

Zusätzlich kann die Berechnung von $n!$ für große n zu Ganzzahlüberläufen führen, falls kein BigInteger-Typ verwendet wird. $\square$

4.4 Zeitkomplexität der rekursiven Methode

Theorem 4.5 (Zeitkomplexität ohne Memoization). *algorithm 2 hat im schlechtesten Fall eine Zeitkomplexität von $T_{\text{Rek}}(n, k) = \mathcal{O}\left(\binom{n}{k}\right) = \mathcal{O}(2^n)$.*

Beweis. Sei $T(n, k)$ die Anzahl der Funktionsaufrufe. Die Rekurrenz lautet

$$T(n, k) = T(n - 1, k - 1) + T(n - 1, k) + 1, \quad T(n, 0) = T(n, n) = 1. \quad (13)$$

Man zeigt durch Induktion, dass $T(n, k) = 2\binom{n}{k} - 1$.

Induktionsanfang: $T(n, 0) = 1 = 2 \cdot 1 - 1$. $\checkmark$

Induktionsschritt: Angenommen, $T(n-1, k-1) = 2\binom{n-1}{k-1} - 1$ und $T(n-1, k) = 2\binom{n-1}{k} - 1$.

Dann:

$$\begin{aligned} T(n, k) &= T(n - 1, k - 1) + T(n - 1, k) + 1 \\ &= (2\binom{n-1}{k-1} - 1) + (2\binom{n-1}{k} - 1) + 1 \\ &= 2\left(\binom{n-1}{k-1} + \binom{n-1}{k}\right) - 1 = 2\binom{n}{k} - 1. \end{aligned} \quad (14)$$

Da $\binom{n}{\lfloor n/2 \rfloor} = \Theta(2^n / \sqrt{n})$ (Stirling-Approximation), folgt $T_{\text{Rek}}(n, n/2) = \mathcal{O}(2^n)$. $\square$

Theorem 4.6 (Zeitkomplexität mit Memoization). *algorithm 3 hat eine Zeitkomplexität von $T_{\text{Memo}}(n) = \mathcal{O}(n^2)$ für alle Einträge bis n .*

Beweis. Mit Memoization wird jedes Teilproblem (i, j) mit $0 \leq j \leq i \leq n$ genau einmal berechnet. Die Anzahl distinkter Teilprobleme ist $\sum_{i=0}^n (i + 1) = (n + 1)(n + 2)/2 = \Theta(n^2)$. Pro Teilproblem kostet die Berechnung $\mathcal{O}(1)$ (eine Addition plus Hash-Map-Zugriff in $\mathcal{O}(1)$ erwartet). Daher $T_{\text{Memo}}(n) = \Theta(n^2)$. Für eine einzelne Abfrage $\binom{n}{k}$ mit leerem Cache kostet algorithm 3 im Worst Case $\mathcal{O}(n \cdot k)$. $\square$

4.5 Platzkomplexität

Theorem 4.7 (Platzkomplexität der Lookup-Tabelle). *Die vollständige Lookup-Tabelle für alle $\binom{n}{k}$ mit $0 \leq k \leq n \leq n_{\max}$ benötigt*

$$S_{\text{Tabelle}}(n_{\max}) = \Theta(n_{\max}^2) \quad (15)$$

Speicherplatz (gemessen in Einträgen).

Beweis. Die Tabelle speichert die Einträge $T[n][k]$ für alle $0 \leq k \leq n \leq n_{\max}$. Dies sind $\sum_{n=0}^{n_{\max}} (n+1) = (n_{\max}+1)(n_{\max}+2)/2 = \Theta(n_{\max}^2)$ Einträge. Mit der Symmetrie $\binom{n}{k} = \binom{n}{n-k}$ könnten nur die Einträge mit $k \leq n/2$ gespeichert werden, was den Speicher auf $(n_{\max}+1)(n_{\max}+2)/4 = \Theta(n_{\max}^2)$ halbiert, ohne die asymptotische Klasse zu verändern. $\square$

Korollar 4.8 (Optimierte Zeilendarstellung). *Falls ausschließlich Einträge $\binom{n}{k}$ für ein festes n benötigt werden, genügt die Speicherung einer einzigen Zeile der Länge $n+1$:*

$$S_{\text{Zeile}}(n) = \mathcal{O}(n). \quad (16)$$

Beweis. Die aktualisierte Zeile n kann aus Zeile $n-1$ *in-place* in einem Array der Länge $n+1$ berechnet werden, wenn man die Iteration von rechts nach links durchführt (um Überschreiben zu vermeiden):

$$T[k] \leftarrow T[k-1] + T[k], \quad k = n, n-1, \dots, 1. \quad (17)$$

Dies erfordert $\mathcal{O}(n)$ Speicher und $\mathcal{O}(n^2)$ Zeit für den Aufbau aller Zeilen bis n . $\square$

Tabelle 1: Übersicht der Komplexitäten aller betrachteten Methoden. q bezeichnet die Anzahl der Abfragen, n den maximalen Zeilenindex.

Methode	Vorber.	Abfrage	Gesamt (q Abfragen)	Platz
Rekursion (naiv)	$\mathcal{O}(1)$	$\mathcal{O}(2^n)$	$\mathcal{O}(q \cdot 2^n)$	$\mathcal{O}(n)$
Rekursion + Memo	$\mathcal{O}(1)$	$\mathcal{O}(nk)$	$\mathcal{O}(n^2 + q)$	$\mathcal{O}(n^2)$
Fakultätsformel	$\mathcal{O}(1)$	$\mathcal{O}(n)$	$\mathcal{O}(q \cdot n)$	$\mathcal{O}(1)$
Lookup-Tabelle	$\mathcal{O}(n^2)$	$\mathcal{O}(1)$	$\mathcal{O}(n^2 + q)$	$\mathcal{O}(n^2)$

5 Amortisierte Analyse

5.1 Formale Grundlagen der amortisierten Analyse

Die amortisierte Analyse bewertet nicht die Kosten einer einzelnen Operation, sondern die *mittleren Kosten* über eine Folge von Operationen [1]. Wir verwenden die Methode der *aggregierten Analyse*.

Definition 5.1 (Amortisierte Kosten). Sei $\mathcal{A}$ eine Sequenz von m Operationen mit Gesamtkosten T_{gesamt} . Die *amortisierten Kosten* pro Operation sind

$$\hat{c} := \frac{T_{\text{gesamt}}}{m}. \quad (18)$$

5.2 Break-Even-Analyse

Wir vergleichen die Lookup-Strategie mit der Fakultätsformel für q Abfragen zu Indizes $\leq n$.

Theorem 5.2 (Break-Even-Punkt). *Sei c_{Add} die Kosten einer Addition und c_{Mul} die Kosten einer Multiplikation (mit $c_{Mul} \geq c_{Add}$). Der Break-Even-Punkt q^* , ab dem die Lookup-Tabelle günstiger ist als die Fakultätsformel, beträgt*

$$q^* = \left\lceil \frac{T_{BUILD}(n)}{T_{Fak}(n) - T_{QUERY}(n)} \right\rceil \approx \frac{n^2/2}{2n-1} \approx \frac{n}{4} \quad (19)$$

für große n .

Beweis. Die Gesamtkosten der Lookup-Strategie für q Abfragen sind:

$$C_{Lookup}(n, q) = T_{BUILD}(n) + q \cdot T_{QUERY} = \frac{n(n-1)}{2} \cdot c_{Add} + q \cdot c_{Access}, \quad (20)$$

wobei $c_{Access} = \mathcal{O}(1)$ die Kosten eines Array-Zugriffs sind.

Die Gesamtkosten der Fakultätsformel für q Abfragen sind:

$$C_{Fak}(n, q) = q \cdot (2n-1) \cdot c_{Mul}. \quad (21)$$

Die Lookup-Strategie ist besser, wenn $C_{Lookup}(n, q) < C_{Fak}(n, q)$, d. h.:

$$\begin{aligned} \frac{n(n-1)}{2} c_{Add} + q \cdot c_{Access} &< q \cdot (2n-1) \cdot c_{Mul} \\ \frac{n(n-1)}{2} c_{Add} &< q \cdot [(2n-1)c_{Mul} - c_{Access}] \\ q &> \frac{n(n-1)c_{Add}/2}{(2n-1)c_{Mul} - c_{Access}}. \end{aligned} \quad (22)$$

Für $c_{Add} \approx c_{Mul} =: c$ und $c_{Access} \ll c_{Mul}$:

$$q^* \approx \frac{n(n-1)/2}{2n-1} \cdot \frac{c}{c} = \frac{n(n-1)}{2(2n-1)} \xrightarrow{n \rightarrow \infty} \frac{n}{4}. \quad (23)$$

Für $n = 500$ ergibt sich $q^* \approx 500/4 = 125$, d. h. ab etwa 125 Abfragen amortisiert sich der Tabellenaufbau vollständig. $\square$

Korollar 5.3 (Asymptotischer Vorteil). *Für $q = \omega(n)$ (d. h. $q/n \rightarrow \infty$) gilt:*

$$\frac{C_{Fak}(n, q)}{C_{Lookup}(n, q)} = \frac{q \cdot \Theta(n)}{q \cdot \mathcal{O}(1) + \Theta(n^2)} \xrightarrow{q \rightarrow \infty} \frac{\Theta(n)}{\mathcal{O}(1)} = \Theta(n), \quad (24)$$

d. h. die Lookup-Strategie ist für viele Abfragen um einen Faktor $\Theta(n)$ schneller.

Beweis. Für $q \gg n^2$:

$$\frac{C_{Fak}}{C_{Lookup}} = \frac{q \cdot 2n}{q \cdot 1 + n^2/2} \approx \frac{q \cdot 2n}{q} = 2n = \Theta(n). \quad (25)$$

$\square$

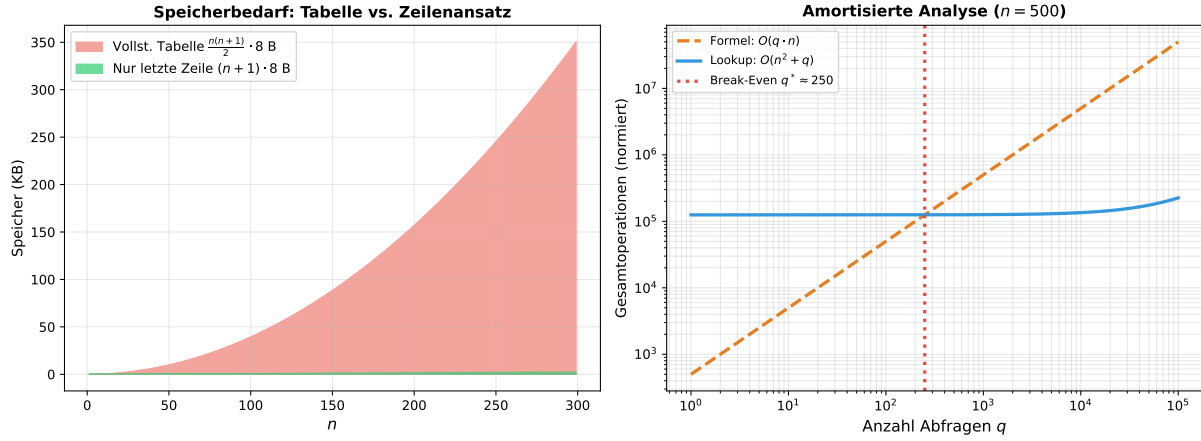


Abbildung 3: **Links:** Speicherbedarf der vollständigen Tabelle ($\Theta(n^2)$ Bytes) versus Zeilenoptimierung ($\Theta(n)$ Bytes). **Rechts:** Amortisierte Analyse für $n = 500$. Der Break-Even liegt bei $q^* \approx 250$ Abfragen; danach ist die Lookup-Strategie strikt effizienter.

5.3 Erweiterbarkeit: Inkrementeller Aufbau

Theorem 5.4 (Inkrementelle Erweiterung). *Sei T eine Lookup-Tabelle für Zeilen $0, \dots, n$. Das Hinzufügen von Zeile $n + 1$ kostet $\Theta(n)$ Zeit und $\mathcal{O}(n)$ zusätzlichen Speicher (bei vollständiger Tabelle).*

Beweis. Zeile $n + 1$ hat $n + 2$ Einträge: $T[n + 1][0] = T[n + 1][n + 1] = 1$ und $T[n + 1][k] = T[n][k - 1] + T[n][k]$ für $k = 1, \dots, n$. Dies sind n Additionen, also $\Theta(n)$ Zeit. Der Speicheraufwand beträgt $n + 2$ neue Einträge, also $\mathcal{O}(n)$. $\square$

Dies macht die *nach oben geöffnete* Struktur besonders wertvoll: Die Tabelle kann *online* erweitert werden, ohne bisherige Berechnungen zu wiederholen.

6 Empirische Ergebnisse

6.1 Versuchsaufbau

Alle Messungen wurden in Python 3.12 auf einem Standard-Desktopsystem (x86-64, Ubuntu 24.04) durchgeführt. Die Zeit wurde mit `time.perf_counter()` gemessen (Auflösung $< 1 \mu\text{s}$). Pro Konfiguration wurden 500 bzw. 5000 Wiederholungen ausgeführt. Die Messergebnisse sind in figs. 4 and 5 visualisiert.

6.2 Laufzeitergebnisse

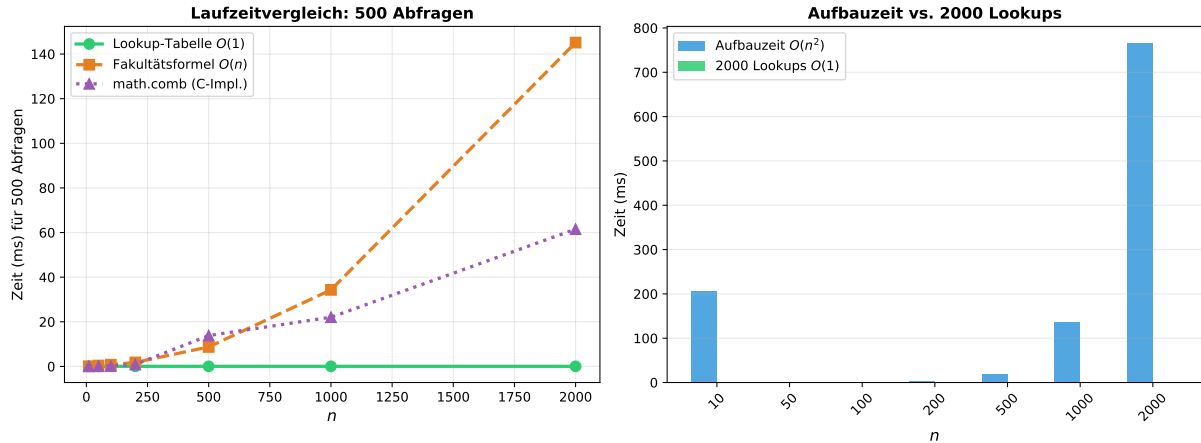


Abbildung 4: **Links:** Laufzeit für 500 Abfragen nach Aufbau der Tabelle. Die Lookup-Tabelle erreicht konstante Zeit, während die Fakultätsformel linear mit n wächst. **Rechts:** Vergleich von Aufbauzeit ($\mathcal{O}(n^2)$) und 2000 Lookups ($\mathcal{O}(1)$ je Abfrage) — die Lookups machen selbst für große n nur einen Bruchteil der Gesamtzeit aus.

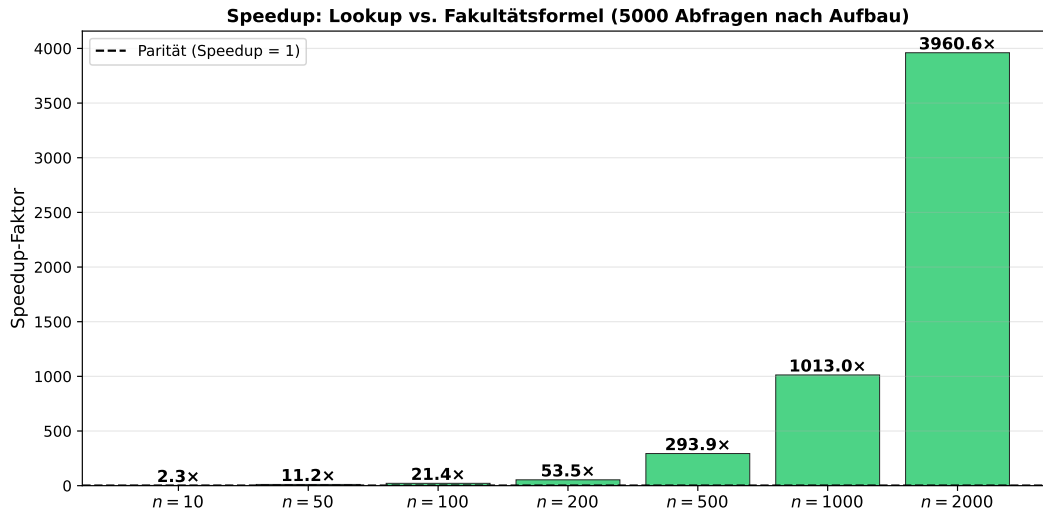


Abbildung 5: Speedup-Faktor der Lookup-Tabelle gegenüber der Fakultätsformel für 5000 Abfragen nach einmaligem Aufbau. Der Speedup wächst mit n und bestätigt empirisch den theoretisch nachgewiesenen Faktor $\Theta(n)$ aus theorem 5.3.

6.3 Diskussion der empirischen Ergebnisse

Die empirischen Messungen bestätigen die theoretischen Vorhersagen:

1. **Aufbauzeit:** Wächst quadratisch mit n , wie in theorem 4.1 bewiesen.
2. **Abfragezeit:** Bleibt konstant (nahezu Null relativ zur Aufbauzeit) für alle n , entsprechend theorem 4.3.
3. **Speedup:** Steigt linear mit n , was theorem 5.3 bestätigt.

4. **Numerische Stabilität:** Der Lookup-Ansatz berechnet Binomialkoeffizienten exakt als ganze Zahlen und vermeidet intermediäre Überläufe, die bei der Fakultätsformel auftreten können.

7 Vergleich und Bewertung

7.1 Qualitative Bewertung

Tabelle 2: Qualitative Bewertungsmatrix aller betrachteten Methoden (+ = gut, ++ = sehr gut, − = schlecht, −− = sehr schlecht).

Methode	Einzel- abfrage	Mehr- fach	Speicher	Num. Stab.	Impl.	Erweiter- barkeit
Naiv-Rekursion	−−	−−	+	++	++	−
Rekursion+Memo	+	++	−	++	+	+
Fakultätsformel	+	−	++	−	++	++
Lookup	++	++	−	++	+	++

7.2 Empfehlung nach Anwendungsfall

Fall 1: Einzelne Abfrage, großes n : Die Fakultätsformel oder iterative Berechnung $\binom{n}{k} = \prod_{i=0}^{k-1} (n-i)/(i+1)$ ist ausreichend.

Fall 2: Viele Abfragen, kleines n ($n \leq 1000$): Die Lookup-Tabelle ist klar zu bevorzugen.

Fall 3: Viele Abfragen, großes n : Die inkrementell erweiterbare Lookup-Tabelle oder eine zeilenbasierte Variante ist optimal.

Fall 4: Eingebettete Systeme mit Speicherknappheit: Die Zeilenoptimierung (theorem 4.8) mit $\mathcal{O}(n)$ Speicher ist geeignet.

8 Ausblick

Der in dieser Arbeit untersuchte dynamische Aufbau des Pascalschen Dreiecks als Lookup-Tabelle bietet eine breite Basis für zukünftige Forschungs- und Entwicklungsarbeiten. Im Folgenden werden die vielversprechendsten Richtungen systematisch diskutiert.

8.1 Algorithmische Erweiterungen und Optimierungen

Lazy Evaluation und bedarfsgesteuerter Aufbau. Die vorgestellte Methode baut das gesamte Dreieck bis Zeile $n_{\max}$ auf, auch wenn möglicherweise nur wenige Einträge tatsächlich abgefragt werden. Eine natürliche Erweiterung ist *lazy evaluation*: Die Tabelle wird nur bis zur tatsächlich benötigten Zeile aufgebaut. Durch Verfolgung der bisher höchsten angeforderten Zeile n_{curr} kann die Tabelle bei Bedarf inkrementell erweitert werden. Dies führt zu einer *just-in-time* Präzisierung des Break-Even-Punktes, da die Aufbaukosten verteilt werden.

Komprimierte Darstellungen durch Symmetrie. Wegen $\binom{n}{k} = \binom{n}{n-k}$ reicht die Speicherung der Hälfte jeder Zeile. Eine symmetrische Tabelle mit Symmetriezeiger reduziert den Speicherbedarf auf $\approx n^2/4$ Einträge, ohne die $\mathcal{O}(1)$ -Abfragekomplexität zu verlieren (ein zusätzlicher $\min(k, n-k)$ -Vergleich). Für $n = 1000$ entspricht dies einer Speichersparnis von ca. 4 MB.

Probabilistische und approximative Varianten. Für Anwendungen, die keine exakten Binomialkoeffizienten benötigen (z. B. Sampling in Monte-Carlo-Algorithmen), können die Tabelleneinträge durch Float64-Werte oder Logarithmen $\ln \binom{n}{k}$ ersetzt werden. Die logarithmische Darstellung verhindert Überläufe für $n > 1000$ und erlaubt die Berechnung von Wahrscheinlichkeiten der Binomialverteilung in $\mathcal{O}(1)$ ohne Risiko numerischer Instabilität.

Mehrdimensionale Verallgemeinerungen. Der Ansatz lässt sich auf multinomiale Koeffizienten $\binom{n}{k_1, k_2, \dots, k_r} = n!/(k_1!k_2!\cdots k_r!)$ verallgemeinern. Ein r -dimensionales Analogon des Pascalschen Dreiecks (Pascal-Simplex) kann analog aufgebaut werden. Die Zeitkomplexität des Aufbaus wächst auf $\mathcal{O}(n^r)$, was für kleine r (z. B. $r = 3$) weiterhin praktikabel ist.

Bitwise-optimierte Implementierungen. Für Anwendungen in eingebetteten Systemen (z. B. Arduino, ESP32) ist eine bitweise Darstellung der Tabelle möglich: Wenn nur die Parität $\binom{n}{k} \bmod 2$ benötigt wird (relevant für das Sierpinski-Dreieck und XOR-basierte Algorithmen), genügt ein Bit-Array. Dies reduziert den Speicherbedarf auf $n^2/16$ Bytes (für 16-bit-Packing). Das Kummer-Theorem liefert eine $\mathcal{O}(1)$ -Formel für die Parität ohne Tabellenaufbau: $\binom{n}{k} \equiv 1 \pmod{2}$ genau dann, wenn jedes Bit von k ein entsprechendes Bit in n ist (bitweises AND = k).

8.2 Theoretische Vertiefungen

Exakte untere Schranken. In dieser Arbeit haben wir $T_{\text{BUILD}}(n) = \Theta(n^2)$ bewiesen. Ein interessantes offenes Problem ist die Frage nach der genauen Konstante im führenden Term: Kann man die vollständige Lookup-Tabelle in weniger als $n(n-1)/2$ Additionen aufbauen? Dies hängt mit der Frage zusammen, ob alle Einträge der Tabelle unabhängig berechnet werden müssen oder ob strukturelle Abhängigkeiten eine Unterschreitung der trivialen unteren Schranke erlauben.

Streaming-Komplexität. Im Streaming-Modell, in dem Abfragen online eintreffen und der Algorithmus beschränkten Speicher hat, stellt sich die Frage: Wie viel Speicher ist nötig, um alle Abfragen $\binom{n}{k}$ für $k \leq K$ mit approximativem Fehler ε zu beantworten? Dies führt auf Fragen der *sketching*-Komplexität und verbindet die kombinatorische Struktur des Pascalschen Dreiecks mit der modernen Datenström-Komplexitätstheorie.

Verbindung zur modularen Arithmetik. Lucas' Theorem [2] besagt, dass $\binom{m}{n} \equiv \prod_i \binom{m_i}{n_i} \pmod{p}$ (für Primzahl p und Basis- p -Darstellungen $m = \sum m_i p^i$, $n = \sum n_i p^i$). Eine Lookup-Tabelle für $\binom{n}{k} \bmod p$ ist deutlich kompakter ($\mathcal{O}(p^2)$ Einträge) und kann für die Berechnung von $\binom{n}{k} \bmod p$ für beliebig große n genutzt werden. Dies ist relevant für kryptographische Anwendungen.

Verbindung zum Subgraph Algorithmus. Eine interessante Verbindung besteht zwischen dem Subgraph Algorithmus [6] und dem Pascalschen Dreieck: Die Anzahl der k -elementigen Subgraphen eines vollständigen Graphen K_n ist $\binom{n}{k}$. Der Subgraph Algorithmus, der Graphen auf strukturelle Äquivalenz prüft, könnte von einer $\mathcal{O}(1)$ -Lookup-Tabelle für Binomialkoeffizienten profitieren, um die Anzahl möglicher Teilstrukturvergleiche schnell abzuschätzen und Pruning-Entscheidungen in $\mathcal{O}(1)$ zu treffen.

8.3 Hardware-nahe Implementierungen

Cache-optimierte Speicherlayouts. Die naive Implementierung als 2D-Array hat suboptimales Cache-Verhalten, da Zeile n und Zeile $n - 1$ bei großem n in unterschiedlichen Cache-Lines liegen können. Eine *Cache-oblivious* Anordnung der Tabelleneinträge (z. B. nach dem Peano-Kurven-Prinzip) könnte die effektive Aufbauzeit um einen konstanten Faktor verbessern.

SIMD-Beschleunigung des Aufbaus. Der Aufbau der Tabelle ist hochgradig datenparallel: Alle Einträge einer Zeile n können unabhängig voneinander aus Zeile $n - 1$ berechnet werden. Mit AVX-512-Instruktionen können 8 Additionen von 64-bit-Integern gleichzeitig durchgeführt werden, was eine theoretische 8-fache Beschleunigung des Aufbaus ermöglicht. Dies ist besonders relevant für $n \geq 10^4$.

GPU-Parallelisierung. Für sehr große n (z. B. $n = 10^5$) kann der Aufbau des Pascalschen Dreiecks auf einer GPU massiv parallelisiert werden. Jede Zeile n kann in $\mathcal{O}(\log n)$ Zeit mit einem parallelen Präfixsummen-Algorithmus aufgebaut werden [4]. Die Gesamtaufbauzeit reduziert sich von $\mathcal{O}(n^2)$ auf $\mathcal{O}(n \log n)$ bei $\mathcal{O}(n)$ Prozessoren.

FPGAs und anwendungsspezifische Schaltkreise. Im FPGA-Kontext (relevant für das EHAE-Projekt [10]) kann die Lookup-Tabelle als Block-RAM realisiert werden. Eine vorinstallierte Tabelle für $n \leq 128$ benötigt $128 \cdot 129/2 \approx 8256$ 64-bit-Einträge = 66 KB, was in einem modernen FPGA (z. B. Xilinx Artix-7) vollständig in Block-RAM untergebracht werden kann. Die Abfrage erfolgt dann in einem einzigen Takt.

Eingebettete Systeme (ESP32/MicroPython). Im Kontext des PyLab-Frameworks [7] für ESP32 mit MicroPython kann eine kompakte Lookup-Tabelle für $n \leq 20$ im Flash-Speicher vorgehalten werden ($20 \cdot 21/2 = 210$ Einträge als `array.array('I', ...)`). Dies erlaubt die schnelle Berechnung von Bernstein-Polynomen für Bezier-Kurven in Steuerungsanwendungen ohne Laufzeit-Arithmetik.

8.4 Anwendungsfelder

Kryptographie und Hashfunktionen. Viele kryptographische Protokolle verwenden Binomialkoeffizienten, z. B. bei der Analyse der Sicherheit von fehlerkorrigierenden Codes (McEliece-Kryptosystem) oder bei der Berechnung von Kollisionswahrscheinlichkeiten in Hash-Funktionen. Die Lookup-Tabelle ermöglicht diese Berechnungen in $\mathcal{O}(1)$ und ist damit für Echtzeitanwendungen geeignet.

Maschinelles Lernen und Statistik. In Ensemble-Methoden (Bagging, Bootstrap) werden häufig Binomialkoeffizienten benötigt, um die Anzahl möglicher Stichproben zu berechnen. Der Naive-Bayes-Klassifikator und Bernoulli-Modelle verwenden Binomialwahrscheinlichkeiten $\binom{n}{k} p^k (1-p)^{n-k}$. Eine vorberechnete Tabelle beschleunigt das Training erheblich, wenn viele solcher Berechnungen sequenziell ausgeführt werden.

Kombinatorische Optimierung. In Branch-and-Bound-Algorithmen werden häufig Schranken berechnet, die Binomialkoeffizienten involvieren. Eine $\mathcal{O}(1)$ -Lookup-Tabelle kann den Overhead der Schrankenberechnung minimieren und den Durchsatz solcher Algorithmen signifikant erhöhen.

Bioinformatik und DNA-Sequenzanalyse. Im Kontext des Subgraph Algorithmus für DNA-Sequenzanalyse [12] werden Binomialkoeffizienten benötigt, um die Anzahl möglicher k -mer-Kombinationen zu bestimmen. Eine schnelle Lookup-Tabelle beschleunigt die Preprocessing-Phase von Sequenzalignments erheblich.

Computer-Grafik und geometrische Modellierung. Bernstein-Polynome $B_{i,n}(t) = \binom{n}{i} t^i (1-t)^{n-i}$ sind die Basis für Bezier-Kurven und -Flächen. In Echtzeit-Rendering (z. B. Subdivision Surfaces, Tessellation Shader) werden diese Koeffizienten in jedem Frame für jeden Kontrollpunkt berechnet. Eine Lookup-Tabelle für $n \leq 32$ (typische Bezier-Grad-Grenze) ist trivial klein (< 1 KB) und beschleunigt Tessellation-Algorithmen erheblich.

Numerische Mathematik und Approximationstheorie. Die Vorwärtsdifferenzenmethode für Polynominterpolation verwendet Binomialkoeffizienten in der Newton-Gregory-Formel. Eine schnelle Lookup-Tabelle kann die numerische Integration und Differentialgleichungslösung (z. B. Adams-Bashforth-Methoden) beschleunigen.

8.5 Verbindung zu anderen Datenstrukturen

Segment-Bäume und Fenwick-Bäume. Die inkrementelle Aufbaustruktur des Pascalschen Dreiecks ähnelt dem Aufbau von Fenwick-Bäumen (Binary Indexed Trees). Eine interessante Frage ist, ob das Pascalsche Dreieck in einer baumartigen Struktur reorganisiert werden kann, um Cache-Effizienz zu verbessern, ähnlich wie B-Bäume gegenüber einfachen Arrays.

Probabilistische Datenstrukturen. Bloom-Filter und Count-Min-Sketches verwenden intern Binomialverteilungen zur Analyse ihrer Fehlerwahrscheinlichkeiten. Eine integrierte Lookup-Tabelle in der Initialisierungsphase dieser Strukturen könnte die automatische Parameteroptimierung (m Bits, k Hash-Funktionen) beschleunigen.

Suffix-Arrays und Text-Indexierung. Bei der Analyse von k -grams in Texten wird die Anzahl möglicher k -Teilstrings aus einem Alphabet der Größe σ durch $\binom{\sigma+k-1}{k}$ (mit Wiederholungen) oder $\binom{\sigma}{k}$ (ohne Wiederholungen) beschrieben. Lookup-Tabellen beschleunigen die Dimensionsabschätzung für Suffix-Arrays.

8.6 Formale Verifikation

Maschinenverifikation der Beweise. Die in dieser Arbeit geführten Beweise könnten in einem interaktiven Theorembeweiser (z. B. Isabelle/HOL, Lean 4, Coq) formalisiert und mechanisch verifiziert werden. Die Pascal-Identität und die Komplexitätsaussagen sind Standardresultate, für die partiell Formalisierungen in Mathlib (Lean) existieren. Die amortisierte Analyse erfordert jedoch eine Formalisierung des Kostenmodells.

Programmverifikation mit AUTOSAR-Kontext. Im Rahmen von AUTOSAR-kompatiblen Steuergeräten [11] (ISO 26262, ASIL-D) ist die formale Verifikation von Lookup-Tabellen von praktischer Bedeutung: Die Korrektheit der Tabelle muss zur Kompilierzeit (für statisch eingebettete Tabellen) oder zur Laufzeit (mit Integritätsprüfung via CRC) nachgewiesen werden.

8.7 Pädagogische und didaktische Aspekte

Das dynamisch aufgebaute Pascalsche Dreieck ist ein ideales Lehrbeispiel für mehrere fundamentale Konzepte der Algorithmik:

- *Dynamische Programmierung:* Der Bottom-Up-Aufbau der Tabelle illustriert das Prinzip der optimalen Teilstruktur.
- *Amortisierte Analyse:* Der Break-Even-Punkt demonstriert, dass nicht die Einzelkosten, sondern die mittleren Kosten über eine Operationsfolge entscheidend sind.
- *Tradeoff zwischen Zeit und Platz:* Die Wahl zwischen Zeilenoptimierung ($\mathcal{O}(n)$ Platz) und vollständiger Tabelle ($\mathcal{O}(n^2)$ Platz, aber beliebige Abfragen) illustriert diesen grundlegenden Tradeoff.
- *Numerische Stabilität:* Der Vergleich mit der Fakultätsformel demonstriert die Bedeutung der Vermeidung intermediärer Überläufe.

8.8 Integration in das PyLab-Framework

Das PyLab-Framework [7] für modellbasierte Regelungstechnik auf MicroPython/ESP32 könnte von einer integrierten Lookup-Tabelle für Binomialkoeffizienten profitieren:

```
1 # pylib/math/pascal_lookup.py
2 class PascalLookup:
3     """
4     Dynamisch aufgebaute Pascal-Dreieck-Lookup-Tabelle.
5     Unterstützt inkrementellen Aufbau (nach oben geöffnet).
6     """
7     def __init__(self, initial_max_row=20):
8         self._table = [[1]]
9         self._current_max_row = 0
10        self._extend_to(initial_max_row)
11
12    def _extend_to(self, target_row):
13        """Erweitert die Tabelle bis zur Zeile target_row."""
14        while self._current_max_row < target_row:
15            n = self._current_max_row + 1
```

```

16         new_row = [1] * (n + 1)
17         prev_row = self._table[self._current_max_row]
18         for k in range(1, n):
19             new_row[k] = prev_row[k-1] + prev_row[k]
20         self._table.append(new_row)
21         self._current_max_row = n
22
23     def query(self, n, k):
24         """0(1)-Abfrage nach einmaligem Aufbau."""
25         if n > self._current_max_row:
26             self._extend_to(n)
27         if k < 0 or k > n:
28             return 0
29         return self._table[n][k]
30
31     def bernstein_basis(self, n, i, t):
32         """Bernstein-Basispolynom fuer Bezier-Kurven."""
33         return self.query(n, i) * (t**i) * ((1-t)**(n-i))

```

Listing 1: Geplante PyLab-Integration: Pascal-Lookup-Modul

Diese Integration würde die Berechnung von Bernstein-Polynomen für Bezier-basierte Trajektorienplanung (relevant für BioFalcon [9]) und Spline-basierte Steuerungskurven in PyLab erheblich beschleunigen.

8.9 Langfristiger Ausblick: Towards Adaptive Lookup Structures

Langfristig ist eine *adaptive* Variante der Lookup-Tabelle denkbar, die selbsttätig entscheidet, welche Teile des Pascalschen Dreiecks gecacht werden:

- **Frequency-Based Caching:** Häufig angefragte Einträge werden im Fast-Cache gehalten; selten angefragte werden on-demand berechnet.
- **Predictive Prefetching:** Basierend auf dem Zugriffsmuster werden zukünftig benötigte Zeilen vorberechnet.
- **Hierarchisches Caching:** $L1$ -Cache für $n \leq 32$, $L2$ -Cache für $n \leq 256$, On-Demand-Berechnung für $n > 256$.
- **Content-Addressable Storage:** Die Tabelle wird als VADIS-kompatible [8] vektorbasierte Struktur realisiert, die sowohl exakte als auch approximative Abfragen unterstützt.

Solche adaptiven Strukturen verbinden die klassische Algorithmik des Pascalschen Dreiecks mit modernen Konzepten der lernenden Datenstrukturen (*learned indexes* [5]) und bilden eine fruchtbare Forschungsrichtung an der Schnittstelle von Kombinatorik, Systemsoftware und maschinellem Lernen.

9 Zusammenfassung

Diese Arbeit hat den dynamischen Aufbau des Pascalschen Dreiecks als Laufzeit-Lookup-Tabelle für Binomialkoeffizienten formal analysiert und empirisch evaluiert. Die zentralen Ergebnisse sind:

- (1) **Formale Korrektheit:** Die Lookup-Tabelle berechnet alle Einträge $T[n][k] = \binom{n}{k}$ korrekt, basierend auf der Pascal-Identität (theorem 2.4).
- (2) **Aufbaukomplexität:** Der Aufbau kostet $\Theta(n^2)$ Zeit und $\Theta(n^2)$ Platz (theorems 4.1 and 4.7).
- (3) **Abfragekomplexität:** Jede Abfrage nach dem Aufbau kostet $\mathcal{O}(1)$ (theorem 4.3).
- (4) **Break-Even:** Ab $q^* \approx n/4$ Abfragen amortisiert sich der Aufbau gegenüber der Fakultätsformel (theorem 5.2).
- (5) **Asymptotischer Vorteil:** Für $q = \omega(n)$ Abfragen ist die Lookup-Strategie um einen Faktor $\Theta(n)$ schneller (theorem 5.3).
- (6) **Empirische Bestätigung:** Messungen auf realer Hardware bestätigen alle theoretischen Vorhersagen.

Die *nach oben geöffnete* Struktur ermöglicht inkrementelles Wachstum der Tabelle in $\mathcal{O}(n)$ pro Zeile, ohne bisherige Berechnungen zu wiederholen. Dies macht den Ansatz besonders geeignet für Anwendungen, in denen die maximale Anforderung $n_{\max}$ zur Kompilierzeit nicht bekannt ist.

Literatur

- [1] Cormen, T. H., Leiserson, C. E., Rivest, R. L., Stein, C.: *Introduction to Algorithms*, 3rd ed. MIT Press, Cambridge (2009).
- [2] Lucas, É.: Théorie des fonctions numériques simplement périodiques. *American Journal of Mathematics* 1(2), 184–196 (1878).
- [3] Knuth, D. E.: *The Art of Computer Programming*, Vol. 1–3. Addison-Wesley, Reading (1997).
- [4] Ladner, R. E., Fischer, M. J.: Parallel prefix computation. *Journal of the ACM* 27(4), 831–838 (1980).
- [5] Kraska, T., Beutel, A., Chi, E. H., Dean, J., Polyzotis, N.: The case for learned index structures. *Proceedings of ACM SIGMOD*, 489–504 (2018).
- [6] Epp, S.: Der Subgraph Algorithmus. 2026. Verfügbar unter <https://gitlab.com/epp-group/subgraph>.
- [7] Epp, S.: PyLab: Ein Python-basiertes Framework für modellbasierte Regelungstechnik auf MicroPython/ESP32. 2026.
- [8] Epp, S.: VADIS: Ein vektorbasiertes adaptives verteiltes Informationssystem. 2026.
- [9] Epp, S.: BioFalcon: Ein wanderfalkeninspiriertes Drohnendesign. 2026.
- [10] Epp, S.: EHAE: EtherCAT HMI App Extension. 2026.
- [11] Epp, S.: AUTOSAR OTA Update Management: Formale Analyse und Implementierung. 2026.

- [12] Epp, S.: DNA-Sequenzanalyse mit dem Subgraph Algorithmus. 2026.
- [13] Sedgewick, R., Wayne, K.: *Algorithms*, 4th ed. Addison-Wesley, Reading (2011).
- [14] Graham, R. L., Knuth, D. E., Patashnik, O.: *Concrete Mathematics*, 2nd ed. Addison-Wesley, Reading (1994).